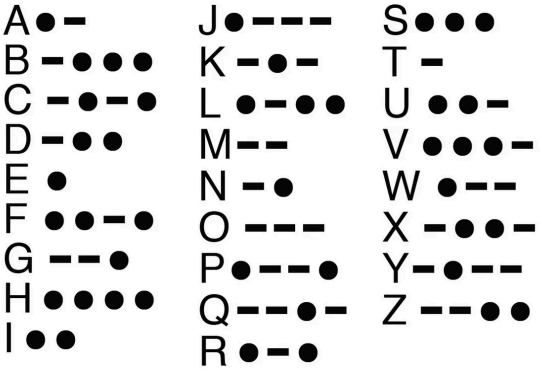
CSU23021-GRP14

Assignment 02 - Creating a Game to Teach Morse Code

20.03.2023 - 09.04.2023

—



<u>Group Members & Roles:</u>

Ingrid Bustnes - GitLab Workflow Owner

Della Doherty - Project Demonstration Owner

Faith Olopade - Project Workflow Owner

Martha Ryan - Project Documentation Owner

Leah Weldon - Project Code Owner



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Introduction

Doxygen Documentation Link: *https://faitholopade.github.io/CSU23021-GRP14/index.html*

For this group project, our main goals were to create a fun and educational Morse Code game as well as fostering good communication and teamwork with all group members. By implementing effective project management techniques, using Git for version control, and creating a well put-together final project report and application demonstration film, we have successfully worked as a team to meet the project's goals.

The Morse Code game was created by combining C programming with ARM assembly language to produce an interactive game on our MAKER-PI-PICO boards. Our game's C code file controls user input, game logic, and level advancement. Our ARM assembly code fine-tunes hardware control for better performance. With the integration of high-level and low-level programming, players have a smooth and engaging gaming experience.

Our game consists of four levels, each with varying difficulty levels. In level one, the player is presented with an individual character either from the alphabet (A-Z) or a number(0-9) and its corresponding morse code. By pushing the GP21 button on the PI-PICO board players can input either dots or dashes. To create a dot, players press the button quickly, to create a dash, the GP21 button is held for longer and a space is created when the button has not been pressed for a short while. For level 2, the player is only presented with a character and no corresponding morse code. For level 3, the player is presented with a word and its corresponding morse code and for level 4 the player is just presented with a word. The player has 3 lives and to progress to the next level, must input 5 correct sequences in a row.

We split up the code equally between all members of the group and to allow efficient collaboration, we used GitHub so all work could be done concurrently and put together easily. In the following report we will illustrate how we broke down the tasks, tracked workflow and how we implemented our program and design decisions we took.

Workflow

High level breakdown/ownership of tasks and how they were tracked. Further information can be found in our individual declaration documents

Week 1 [13th Mar - 19th Mar]

Name	**Task**
Ingrid	- Setup GitHub - Created Branches in GitHub - Research
Faith	- Created & Assigned Tasks Using GitHub Issues - Research
Martha	- Created Documentation Template - Research
Leah	- Research
Della	- Research

Week 2 [20th Mar - 26th Mar]

Name	**Task**
Ingrid	- V1.0 Implementation of Requirement 4 Completed - Managed Pull Requests - Created V1.0 Testing Branch
Faith	- V1.0 Implementation of Requirement 6 & 7 Completed - Updated GitHub Issues & Milestones - Divided Documentation Tasks
Martha	- V1.0 Implementation of Requirement 2 & 3 Completed - Updated Documentation
Leah	- V1.0 Implementation of Requirement 5 Completed - Tested V1.0 Implementation

Della	- V1.0 Implementation of Requirement 1 Completed - Tested V1.0 Welcome Message implementation

Week 3 [27th Mar - 2nd Apr]

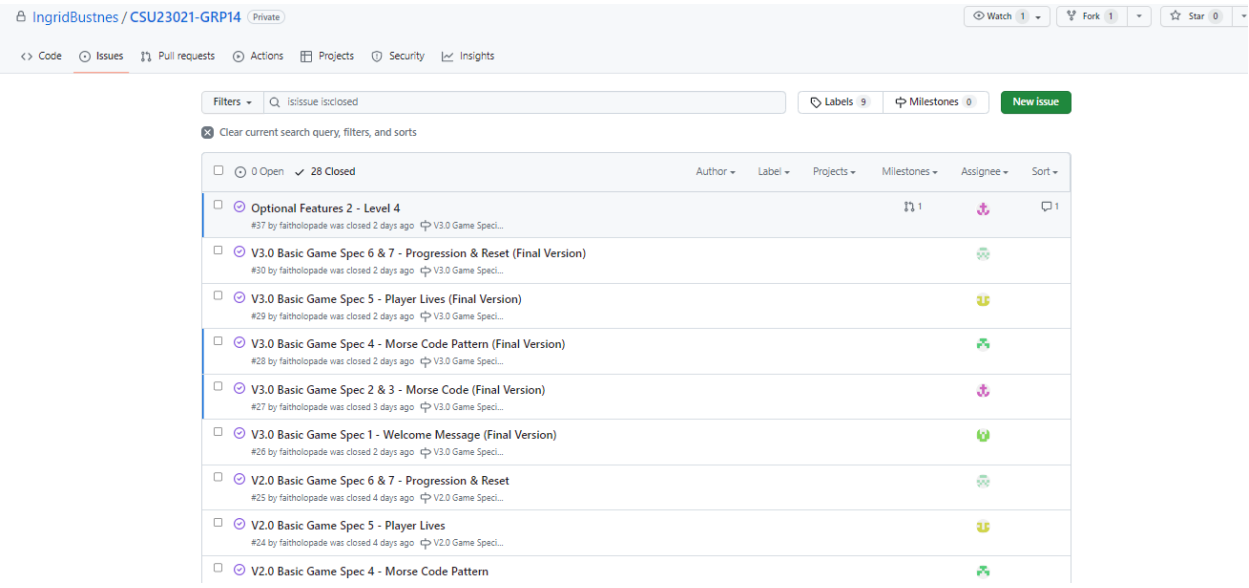
Name	**Task**
Ingrid	- V2.0 Implementation of Requirement 4 Completed - Managed Pull Requests - Created V2.0 Testing Branch - Implementing Statistics Function
Faith	- V2.0 Implementation of Requirement 6 & 7 Completed - Code Review - Updated GitHub Issues & Milestones
Martha	- V2.0 Implementation of Requirement 2 & 3 Completed - Updated Documentation - Implementing Level 4
Leah	- V2.0 Implementation of Requirement 5 Completed - Code Cleanup - Tested V2.0 Implementation
Della	- V2.0 Implementation of Requirement 1 Completed - Implementing Level 3

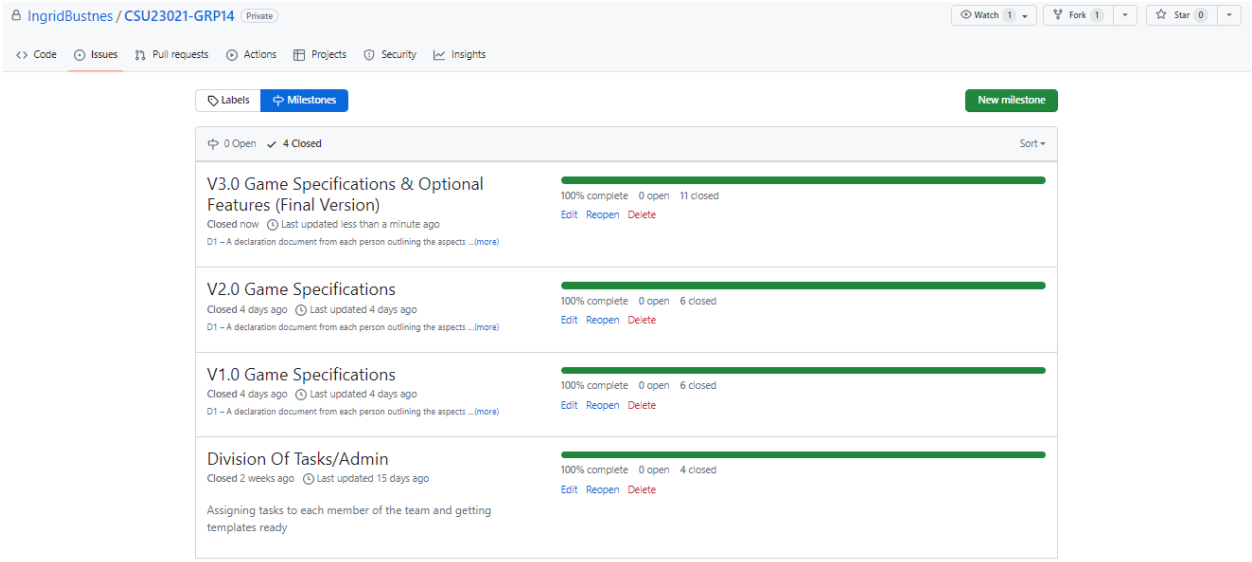
Week 4 [3rd Apr - 9th Apr]

Name	**Task**
Ingrid	- V3.0 Implementation of Requirement 4 Completed - Managed Pull Requests - Created V3.0 Testing Branch - Implemented Statistics Function
Faith	- V3.0 Implementation of Requirement 6 &

	7 Completed - Code Review - Updated GitHub Issues & Milestones
Martha	- V3.0 Implementation of Requirement 2 & 3 Completed - Implemented Level 4 - Doxygen Documentation - Updated Documentation
Leah	- V3.0 Implementation of Requirement 5 Completed - Code Cleanup - Code Documentation - Tested V3.0 Implementation
Della	- V3.0 Implementation of Requirement 1 Completed - Implemented Level 3 - Video Demonstration

Task Management With GitHub Issues & Milestones




IngridBustnes / CSU23021-GRP14
Private
Watch 1
Fork 1
Star 0
Code
Issues
Pull requests
Actions
Projects
Security
Insights
Labels
Milestones
New milestone
0 Open
4 Closed
Sort
V3.0 Game Specifications & Optional Features (Final Version)
Closed now
Last updated less than a minute ago
D1 – A declaration document from each person outlining the aspects ...(more)
100% complete 0 open 11 closed
Edit Reopen Delete
V2.0 Game Specifications
Closed 4 days ago
Last updated 4 days ago
D1 – A declaration document from each person outlining the aspects ...(more)
100% complete 0 open 6 closed
Edit Reopen Delete
V1.0 Game Specifications
Closed 4 days ago
Last updated 4 days ago
D1 – A declaration document from each person outlining the aspects ...(more)
100% complete 0 open 6 closed
Edit Reopen Delete
Division Of Tasks/Admin
Closed 2 weeks ago
Last updated 15 days ago
Assigning tasks to each member of the team and getting templates ready
100% complete 0 open 4 closed
Edit Reopen Delete

GitLab

GitHub Link: *https://github.com/IngridBustnes/CSU23021-GRP14*

Week 1 [13th Mar - 19th Mar]

In week 1 we decided to work on the assignment using Github instead of GitLab as GitHub offers additional features for students such as graphical representations of commit history and a simpler user interface. Ingrid, our GitLab workflow owner, set up a Github Repository and added all members. Faith, as the Project workflow owner had split the requirements from the project description into six different tasks and made these requirements into separate branches and issues in our GitHub repository. The group members were then assigned 1-2 issues each, and started working on their assigned tasks in the respective branch according to their task assignment.

Week 2 [20th Mar - 26th Mar]

In week 2 everyone began working on V1.0 of their individual tasks using a new branch for each task. This allowed for easy implementation without the worry of overwriting someone else's code. As tasks were completed each individual marked the issue relating to their task as completed. This allowed us to keep track of what was left to do and manage the project timeline. At the end of the week we created a testing branch. Everyone opened pull requests to merge their V1.0 code to this branch and Ingrid approved these requests and managed merging. Leah carried out the tests on the code and the working V1.0 implementation was pushed to the master branch as a starting point for V2.0 implementation.

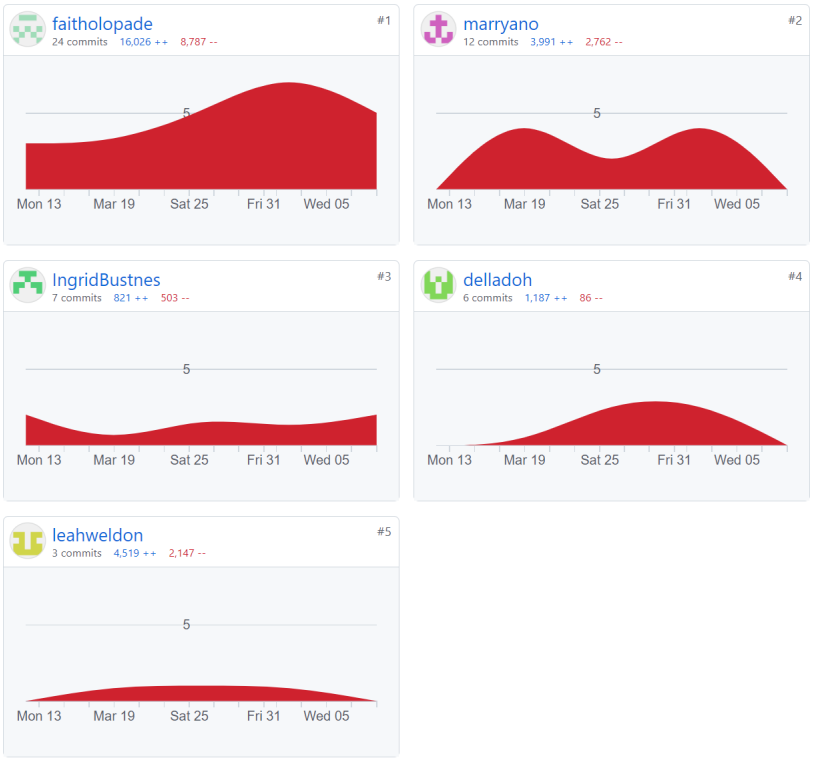
Week 3 [27th Mar - 2nd Apr]

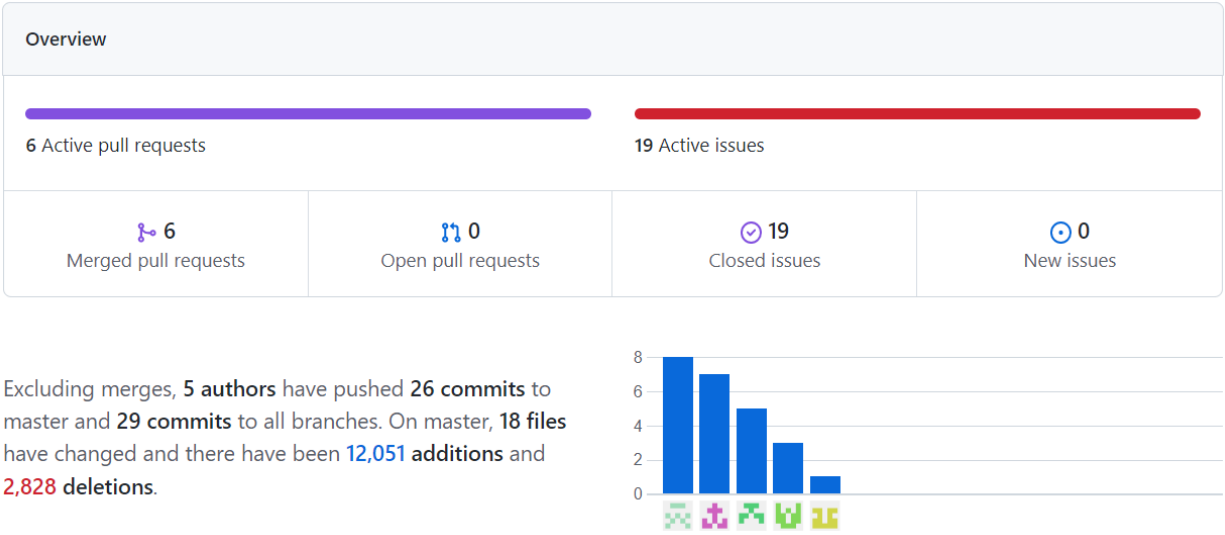
During the third week, our team successfully developed version 2.0 of the standard two-level game. Each team member contributed by refining the code from version 1.0 in their respective branches. Ingrid, our GitHub Workflow Manager, was responsible for merging these branches for testing purposes. To maintain the integrity of the main branch, we established a dedicated testing branch for the merged code. Leah, the designated Code Owner, then thoroughly tested the updated code within this testing branch. By the conclusion of week 3, we had a fully functional game with both Level 1 and Level 2 implemented accurately.

Week 4 [3rd Apr - 9th Apr]

During week 4, our team focused on developing the final version, V3.0, which included the addition of two new levels and summary statistics for each level. We streamlined the development process by creating three separate branches: one for the statistics feature, another for Level 3, and a third for Level 4. Upon completion of these enhancements, we initiated pull requests and subsequently merged them into a consolidated testing branch. After thoroughly testing the integrated features within the testing branch, we confidently updated the main branch with our final, polished code, marking the successful completion of the group project.

GitHub Insights





Code

Week 1 [13th Mar - 19th Mar]

In week 1 we discussed the idea of implementing 2 arrays. One which would hold the morse code and one which would hold the letters A-Z and numbers 0 - 9. An idea we had was to store the letters/numbers at the same index in array 1 as the morse code equivalent in array 2. This would make it easier for us down the line when we begin to compare the player's input sequence with the correct morse code sequence. We mainly focused on brainstorming different approach ideas during week 1 and coding began in week 2. Week one also consisted of assigning the various roles to each of the members and dividing up the basic game specifications between each member so we each knew what coding task we were to tackle in week 2.

Week 2 [20th Mar - 26th Mar]

In week 2 we got our first prototype of our code working with two of the levels implemented. We initialised a watchdog timer to automatically reset our game if an individual fails to press the G21 button and prevent it from falling into an unresponsive state. We implemented a welcome page to print the message “PLAY MORSE” with the various levels listed below. This was created simply in a void function with multiple printf statements. There are functions of the RGB LED functionality, initialising the LED to blue and the function takes in an integer value representing the number of lives remaining in the game and updates the LED colour accordingly using a switch statement. We record the button press duration in a separate function to record the time in milliseconds which is used to determine if the user would like a line or dash. We defined a data structure called ‘morse’ using C which contains morse code, alphanumeric characters, and words associated with the alphanumeric characters. We initialised an array of 36 morse structs that included the morse representation of each letter of the alphabet and the numbers 0-9. We set the initial values of various variables such as the number of lives, sequence and correct answers. We are able to take in user input and return the selected game through another function. For Level one we ask the user to input the morse code equivalent of a given numeric character that we printed on the screen. Our level two is similar to our level one, but we do not provide the morse code equivalent for them to copy. We have various different functions to handle the different levels of the game. We check if the player can move to the next level based on the number of correct answers, remaining lives, and completed levels. We create a function to initialise the user input array to store the morse code entered by the player and we call the display_input() function to display the code that was imputed and we also call on the RGB_update() function to change the LED depending on the number of remaining lives. We created multiple other functions to help with the gameplay such as a function to compare user input with the expected morse code sequence for the current letter of the alphabet. We check if the player has any lives left and if there is at least one life remaining the game will continue. When a player completes a level we print a message indicating that it has been successfully completed and then update the number of successfully completed levels. Another two of our created functions can print either a winning message to congratulate the player or a losing message to express condolences depending on the outcome of the game. We knew we had to incorporate some error handling if the player either failed to input anything or if they failed to input the correct answer so if an incorrect answer is inputted we display an error message.

We each coded a different aspect of the code and merged them to their respective branches before we merged all the code into a version 1 implementation testing branch. This is where any conflicts were resolved and the debugging was carried out along with peer programming to improve the quality of our code and enhance our collaboration as a team.

Week 3 [27th Mar - 2nd Apr]

In Week 3 we finalised the basic requirements of the code. We finished debugging our first implementation and once we were happy with the results we started on the extra features we could add. We added two more levels to our game and included an extra feature to tell the user the success rate of each level they played with another function that divided the total number of incorrect and correct answers by the number of correct answers and obtained a percentage. We added more print messages to indicate completed levels and various other updates such as number of remaining lives and printing out the correct sequence. These extra additions help to optimise our code and improve user experience. For our level 3, we present the user with a word and its corresponding morse code and we prompt the user to enter the morse code for that specific word. We have three different words in this level so using a switch statement we can cycle through these various words. Our level four presents the user with a word in the English language but does not provide its corresponding morse code. Another switch statement is in play here to cycle through the words again. A version 2 implementation texting branch was created where we could merge all of the new code additions and carry out the clean up of our code and handle the various errors we were receiving. We tested the code using this branch to ensure the code worded accordingly.

Week 4 [3rd Apr - 9th Apr]

In Week 4 we essentially had all of her code written and completed but we just needed to finalise the extra features and make the final improvements to our code. We used a version 3 implementation testing branch to thoroughly test the code and to run through each scenario that a user may encounter to ensure that each situation was handled correctly and the correct output was given. We made all of the various different editions to these extra features worked and we did a final code cleanup to ensure that the layout was presentable, readable, and understandable. We made sure to handle any errors that occurred in a timely manner and communicate together each time you're making a commit to this final implementation so that it could be retested to guarantee it was still working smoothly.